BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 8

Array Part 2

Table of Contents

SL No	Topic		Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction		-	-	3
	1.1	Objectives	-	-	
2	Sorting		-	-	4-8
	2.1	Bubble Sort	-	-	
	2.2	Selection Sort	-	-	
	2.3	Insertion Sort	-	1	
3	Multidimensional Arrays		-	2	9-13
4	Summary		-	-	14
5	Terminal Questions		-	-	15
6	Answers To Self Assessment Questions		-	-	15
7	Answers To Terminal Questions		-	-	15-17

1. INTRODUCTION

Sorting is a fundamental operation in computer science used to arrange elements of a list or array in a specific order. Sorting in C is vital for problem-solving, performance optimization, understanding algorithms, laying foundations for advanced topics, real-world applications, and educational purposes, enhancing problem-solving skills, and optimizing performance. In this Unit, we will explore the basics of sorting algorithms like Bubble Sort, Selection Sort, and Insertion Sort, as well as delve into the concept of multi-dimensional arrays, including their declaration, initialization, and memory layout.

1.1. Objectives:

At studying this unit, you should be able to:

- *Understand fundamental sorting algorithms like Bubble sort, Selection Sort, and Insertion sort*
- *Comprehend the concept of multi-dimensional arrays and their applications in representing complex data structures.*
- *Declare and initialize multi-dimensional arrays in C to store and manipulate data efficiently.*

2. SORTING

Sorting in C involves arranging elements of an array or list in a specific order, typically ascending or descending. Common sorting algorithms like Bubble Sort, Selection Sort, and Insertion Sort are implemented to achieve this. Sorting in C enhances problem-solving skills, improves algorithmic understanding, and optimizes performance in various applications. Sorting algorithms can be implemented in C using functions or procedures. These implementations typically involve loops, conditional statements, and array manipulation techniques. It's important to consider factors such as efficiency, stability, and space complexity when choosing a sorting algorithm for a specific task.

Applications: Sorting algorithms find applications in various domains, including:

- Database systems for sorting and indexing records.
- Operating systems for process scheduling and resource allocation.
- Web applications for sorting and displaying data in user interfaces.
- Scientific computing for analyzing and visualizing datasets.

Typical sorting methods are Bubble Sort, Selection Sort, and Insertion Sort.

2.1. Bubble Sort

Bubble Sort is a comparison-based sorting algorithm that iterates through an array multiple times. During each iteration, it compares adjacent elements in the array and swaps them if they are in the wrong order. This process is repeated until the entire array is sorted. The algorithm gets its name from the way smaller elements "bubble up" to their correct positions at the beginning of the array, while larger elements "sink down" to the end. At the end of each iteration, the largest unsorted element is guaranteed to be in its correct position, which reduces the size of the unsorted portion of the array by one element. Here's a step-by-step explanation of the Bubble Sort algorithm:

- Start from the first element (index 0) of the array.
- Compare each element with its adjacent element.
- If the current element is greater than the next element, swap them.

- Continue this process until the end of the array is reached.
- After the first iteration, the largest element will be at the end of the array.
- Repeat steps 1-5 for the remaining elements of the array, excluding the last sorted element.
- Continue these iterations until no more swaps are needed, indicating that the array is sorted.

Implementation:

```
void bubbleSort(int arr[], int n) {
   int i, j;
   for (i = 0; i < n-1; i++) {
      for (j = 0; j < n-i-1; j++) {
         if (arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
         }
      }
   }
}
```

2.2. Selection Sort

Selection Sort is a comparison-based sorting algorithm that divides the input array into two parts: a sorted subarray and an unsorted subarray. It repeatedly selects the smallest (or largest) element from the unsorted subarray and swaps it with the first element of the unsorted subarray. This process continues until the entire array is sorted.

Here's a simplified explanation of the Selection Sort algorithm in C:

Iterate through the array: Start from the first element and iterate through the array.

Find the minimum element: In each iteration, find the smallest element in the unsorted part of the array.

Swap with the first unsorted element: Swap the minimum element with the first unsorted element.

Move the boundary: Move the boundary of the sorted subarray one element ahead.

Repeat: Repeat steps 2-4 until the entire array is sorted.

Implementation:

```
void selectionSort(int arr[], int n) {
   int i, j, min_idx;
   for (i = 0; i < n-1; i++) {
      min_idx = i;
      for (j = i+1; j < n; j++) {
         if (arr[j] < arr[min_idx]) {
            min_idx = j;
         }
      }
      int temp = arr[min_idx];
      arr[min_idx] = arr[i];
      arr[i] = temp;
   }
}
```

2.3. Insertion Sort

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time. It iterates through the array, removing one element from the input data, and inserting it into the correct position in the sorted part of the array. This process repeats until the entire array is sorted.

Here's a simplified explanation of the Insertion Sort algorithm in C:

- Iterate through the array: Start from the second element (index 1) and iterate through the array.
- Compare and insert: For each element, compare it with the elements before it in the sorted subarray.
- If the current element is smaller (or larger) than the elements before it, shift those elements to the right (or left) to make space for the current element.
- Insert the current element into the correct position in the sorted subarray.
- Repeat: Repeat steps 2 for each element in the array until the entire array is sorted.

Insertion Sort is efficient for small datasets or nearly sorted arrays but becomes less efficient as the dataset size increases. Despite its inefficiency for large datasets, Insertion Sort is often used in practice for its simplicity and stability.

Implementation:

```
void insertionSort(int arr[], int n) {
   int i, key, j;
   for (i = 1; i < n; i++) {
      key = arr[i];
      j = i - 1;
      while (j >= 0 && arr[j] > key) {
         arr[j + 1] = arr[j];
         j = j - 1;
      }
      arr[j + 1] = key;
   }
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. Which sorting algorithm repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order?
   (a) Bubble Sort
   (b) Selection Sort
   (c) Insertion Sort
2. Which sorting algorithm divides the input list into two parts: a sorted art at the beginning and an unsorted part at the end?
   (a) Bubble Sort
   (b) Selection Sort
   (c) Insertion Sort
3. Which sorting algorithm builds the final sorted array one element at a time by repeatedly picking the next element and inserting it into the correct position in the sorted part?
   (a) Bubble Sort
   (b) Selection Sort
   (c) Insertion Sort

INSPIRED BY LIFE

3. MULTIDIMENSIONAL ARRAYS

In C, multi-dimensional arrays are used to store data in more than one dimension. They are commonly used to represent matrices, tables, grids, and other structured data. The C language allows arrays of any dimension to be defined.

Declaring and Initializing Multi-dimensional Arrays:

Declaration:

```
// 2D Array Declaration and Initialization
int arr_2d[3][3] = {{0, 1, 2}, {3, 4, 5}, {6, 7, 8}};
// 3D Array Declaration and Initialization
int arr_3d[2][2][2] = {{{0, 1}, {2, 3}}, {{4, 5}, {6, 7}}};
```

Accessing Elements:

```
// Accessing elements in a 2D array
printf("%d", arr_2d[1][2]);  // Outputs: 5
// Accessing elements in a 3D array
printf("%d", arr_3d[0][1][0]);  // Outputs: 2
```

In this section, we will take a look at two-dimensional arrays. One of the most natural applications for a two-dimensional array arises in the case of a matrix. In C, the two-dimensional matrix can be declared as follows:

```
int array[3][6];
```

Following is the way of declaring as well as initializing two-dimensional arrays.

```
int array[3][6] = {
```

```
{4,5,6,7,8,9},

{1,5,6,8,2,4},

{0,4,4,3,1,1}

};
```

Such arrays are accessed like so:

```
array[1][4]= -2;

if (array[2][1] > 0) {

printf ("Element [2][1] is %d", array[2][1]);

}
```

Remember that, like ordinary arrays, two-dimensional arrays are numbered from 0. Therefore, the array above has elements from array[0][0] to array[2][5].

Program: Program to add two matrices

```
#include <stdio.h>

int main()
```

```
{
int a[5][5], b[5][5], c[5][5];
int i, j, m, n;
printf("Enter the order of the matrices:");
scanf("%d%d", &m, &n);
printf(" Enter the elements of A matrix:\n");
for(i=0;i<m;i++)
for(j=0;j<n;j++)
scanf("%d", &a[i][j]);
printf("Enter the elements of B matrix:\n");
for(i=0;i<m;i++)
for(j=0;j<n;j++)
scanf("%d", &b[i][j]);
/* Add the matrices */
for(i=0;i<m;i++)
for(j=0;j<n;j++)
c[i][j] = a[i][j]+b[i][j];
/* Print the sum */
printf("The sum of matrices:\n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
printf("%d\t", c[i][j]);
printf("\n");
}
}
```

Multidimensional arrays are processed in the same manner as one-dimensional arrays, on an element-by-element basis. However, some care is required when passing multidimensional arrays to a function. In particular, the formal argument declarations within the function definition must include explicit size specifications in all of the subscript

positions except the first. These size specifications must be consistent with the corresponding size specifications in the calling program. The first subscript position may be written as an empty pair of square brackets, as with a one-dimensional array. The corresponding function prototypes must be written in the same manner. But while calling the function the array name may be passed as the actual argument as in the case of one-dimensional arrays. E.g:

void process_array (int [][6]); /* function prototype */ void process_array (int array[][6])/*function definition */ {

…

}

Memory Layout of Multi-dimensional Arrays:

Description: In memory, multi-dimensional arrays are stored linearly. For example, in a 2D array, the elements of each row are stored in contiguous memory locations.

Example:

[0, 1, 2]

[3, 4, 5]

[6, 7, 8]

In memory, this 2D array might be represented as [0, 1, 2, 3, 4, 5, 6, 7, 8], where each row is stored continuously.

This means that the elements of the first row (0, 1, 2) are stored first, followed by the elements of the second row (3, 4, 5), and then the elements of the third row (6, 7, 8).

The linear storage of multi-dimensional arrays allows for efficient access to elements using pointer arithmetic and indexing. It also enables cache-friendly memory access patterns, which can improve performance in algorithms that traverse multi-dimensional arrays.

SELF-ASSESSMENT QUESTIONS - 2

4. In a two-dimensional matrix, the first subscript in the declaration specifies number of _____.

5. A two-dimensional array is considered as an array of one-dimensional arrays. (True/False)

6. How are multi-dimensional arrays stored in memory?

   (a) Randomly
   (b) Linearly
   (c) Dispersed

7. In a 2D array, where are the elements of each row stored in memory?

   (a) Non-contiguously
   (b) Dispersed
   (c) Contiguously

8. If a 2D array has dimensions 3x3, how many contiguous memory locations are required to store it?

   (a) 3
   (b) 6
   (c) 9

4. SUMMARY

Sorting algorithms like Bubble Sort, Selection Sort, and Insertion Sort are essential techniques for arranging array elements. Each algorithm offers insights into algorithmic principles, varying in implementation complexity and efficiency.

Multi-dimensional arrays, including 2D and 3D arrays, enable the storage of data in structured formats, such as matrices or grids. Understanding their declaration, initialization, and memory layout is crucial. Multi-dimensional arrays store elements linearly in contiguous memory, enhancing access efficiency. Mastery of these concepts is fundamental for effective data organization, algorithmic problem-solving, and memory management in C programming. Proficiency in sorting algorithms and multi-dimensional arrays provides a solid foundation for handling diverse programming tasks and optimizing code performance.

5. TERMINAL QUESTIONS

1. Write a C program that sorts a 1D array of integers using the Insertion Sort algorithm.
2. Discuss the memory layout of multi-dimensional arrays in C.
3. How does Bubble Sort differ from Selection Sort and Insertion Sort

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) Bubble sort
2. (b) Selection sort
3. (c) Insertion sort
4. Rows
5. True
6. (b) Linearly
7. (c) Contiguously
8. (c) 9

7. ANSWERS TO TERMINAL QUESTIONS

1.
```
#include <stdio.h>

// Function to perform Insertion Sort

void insertionSort(int arr[], int n) {

  int i, key, j;

  for (i = 1; i < n; i++) {

    key = arr[i];

    j = i - 1;


    // Move elements of arr[0..i-1], that are greater than key, to one position
ahead of their current position

    while (j >= 0 && arr[j] > key) {

      arr[j + 1] = arr[j];
```

```
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Call the insertionSort function to sort the array
    insertionSort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
    return 0;
}
```

2. (Refer Section 3 for more details)
3. (Refer Section 2 for more details)